

Basics4AI — Module Content Lifecycle Guide

Status: Design reviewed and approved. **Not yet applied to code.** This guide will be executed together, step by step, on a disposable test branch/environment before touching the live pilot deployment (see the project plan, Step 3).

Who this is for

Anyone who needs to hide, re-enable, add, or replace a module's content (MCQ assessment, SCCCES survey, SIMS survey) in Basics4AI — without needing to understand the whole codebase. Each procedure below is a small, reversible, isolated change.

Background: how the platform decides what modules exist

Basics4AI does **not** keep a single hardcoded list of "the 7 modules" for the student-facing dashboard. Instead, on every app startup it scans the folder `modules/definitions/` for files named `moduleN_definition.py` (plus `pre_course_definition.py` and `post_course_definition.py`), and builds the Student Dashboard from whatever it finds there. This is handled by `modules/registry/discover.py`.

Each module also declares its own position in the sequence via a number called `meta["order"]` inside its definition file (Module 1 has `order: 1`, Module 2 has `order: 2`, and so on; Pre-Course is `order: 0`; Post-Course is `order: 999`, meaning "always last"). A module unlocks for a student once the module immediately before it (by this order) is complete — this logic lives in `core/progress_engine.py` and has no hardcoded reference to any specific module number.

This is why the procedure below works cleanly: removing a module from what gets discovered doesn't just hide its tile — it also closes the gap in the sequence, so whatever module (or Post-Course) came after it now unlocks based on whatever now comes immediately before it.

Procedure 1 — Hide / disable a module (e.g., Module 7 for this pilot)

This makes a module disappear from the **Student Dashboard only**. Nothing is deleted. It stays fully visible to Teachers/Admins, and its content (question bank, definitions, scoring rules) is untouched on disk.

The change (2 small edits)

1. **modules/registry/discover.py** — inside `discover_module_definitions()`, add a check that skips any definition whose `meta["status"]` is not "active". Every module definition already has a "status" field (it currently just says "active" for all of them and is never checked — this teaches the code to actually respect it) with a log line noting which module was skipped and why, so a hidden module is never a silent mystery to someone debugging later.
2. **modules/definitions/module7_definition.py** — change one value:

```
"status": "active"
```

to

```
"status": "disabled"
```

That's it. Nothing else needs to change.

What happens automatically as a result

- Module 7's tile disappears from the Student Dashboard.
- **Post-Course automatically unlocks after Module 6 instead of Module 7.** This is a direct, confirmed consequence of the ordering logic described above — no separate change is needed for this.
- Module 7's question bank (`content_dev/module7_question_bank.json`), scoring rules (`streamlit_app/surveys/module7_content_mcq_assessment_scoring.yaml`), and renderer (`modules/module_7.py`) all remain exactly as they were.

What does not change (read this before you rely on it)

- **Module 7 still appears in the Teacher Dashboard and analytics.** The analytics code (`core/analytics/datasets/canonical_loader.py`) and the Teacher Dashboard (`streamlit_app/dashboards/teacher_dashboard.py`) build their module lists independently and don't consult the same discovery mechanism the Student Dashboard uses. Practically: any report that automatically loops over all modules (for example, the IRT psychometric report) will still show a "Module 7" section — it will just say something like "insufficient data," because zero students will have answered it. **This is expected, not a bug.** Fixing this disconnect so all dashboards reflect the same active-module list is a separate, larger piece of work, intentionally deferred (see the Programmer's Guide's "Future Work" section once written).
- The dataset your analytics pipeline builds is still, under the hood, structured as a 7-module dataset with one empty module — not a "true" 6-module dataset. Worth knowing if you're describing your data provenance in a dissertation methods section.

Re-enabling Module 7 later

Change `"status": "disabled"` back to `"status": "active"` in `modules/definitions/module7_definition.py`. Nothing else to undo. The Post-Course unlock threshold will automatically shift back to "after Module 7" the next time the app restarts.

Procedure 2 — Add a brand-new module (e.g., Module 8, or Module 16 for a larger course)

To introduce module N, create these files, following the exact pattern already used by modules 1–7:

File	Purpose
<code>modules/definitions/module{N}_definition.py</code>	Declares the module's metadata, order, and which instruments (MCQ, SCCCES, SIMS) it uses
<code>modules/module_{N}.py</code>	The code that actually renders the module on screen. Must expose a <code>render(username)</code> function and a <code>MODULE_ID</code> constant that exactly matches the <code>module_id</code> used in the definition file above
<code>content_dev/module{N}_question_bank.json</code>	The MCQ question bank for this module
<code>streamlit_app/surveys/module{N}_content_mcq_assessment_scoring.yaml</code>	The answer key / scoring rules for that MCQ
<code>content_dev/learning_objectives.yaml</code>	Optional — add an entry describing the new module's learning objectives, for documentation purposes only (doesn't affect runtime behavior)

Once these files exist, the Student Dashboard picks up the new module **automatically** — no registration step needed, because of the auto-discovery behavior described above.

Important limitation: the Teacher Dashboard and analytics pipeline will **not** automatically know about the new module. Two files currently have the number of modules "baked in" as a fixed range and would need to be manually updated to include module N: `core/analytics/datasets/canonical_loader.py` and `streamlit_app/dashboards/teacher_dashboard.py`. Making this automatic is the deferred "dynamic

dashboards" work mentioned above — until that's done, adding a module to the Student Dashboard and adding it to the Teacher/analytics views are two separate manual steps.

Procedure 3 — Replace a module's content

Replacing the MCQ question bank for module N: edit `content_dev/module{N}_question_bank.json` (the questions) and, if the correct answers change, `streamlit_app/surveys/module{N}_content_mcq_assessment_scoring.yaml` (the answer key). This is fully isolated to that one module — no other module is affected.

Replacing SCCCES or SIMS survey content: these are **not** per-module files. There is exactly one SCCCES survey definition (`streamlit_app/surveys/b4ai_sccces_survey.yaml` + its scoring file) and one SIMS survey definition (`streamlit_app/surveys/b4ai_sims_survey.yaml` + its scoring file) for the **entire course**. Every module simply re-administers the same shared survey. **Editing these files changes the survey for every module that uses it, not just one** — there's no way to give Module 3 a different SCCCES survey than Module 5 without a larger structural change. If you only intend to change one module's survey content, stop and reconsider — you likely want the MCQ question bank instead.

Quick reference

I want to...	Files to touch	Affects other modules?
Hide a module from students (reversibly)	<code>discover.py</code> (once) + that module's status field	No
Re-enable a hidden module	That module's status field only	No
Add a new module	4 new files (see Procedure 2)	No (student view); Teacher/analytics need manual update
Change a module's MCQ questions/answers	That module's question bank + scoring YAML	No
Change SCCCES or SIMS survey questions	The single shared SCCCES/SIMS YAML pair	Yes — every module